

Conditional Failure at the Root Level

Design, Development, and Team Roadmap

Document ID	OAL-CFTRL-DEV-001
Version	0.1-PPLX
Date	September 23, 2026
Architect of Record	Nathan Lewis
Prepared by	Perplexity AI Research Layer
Status	DRAFT — UNDER REVIEW — NOT CANON — ONESTAMP PENDING

Research draft. No statement becomes active canon unless approved by the Architect through ONESTAMP.

Document control

Field	Entry
Scope	Program plan for moving CFTRL from source preservation and concept definition to modeling, validation, governance, and operational use.
Authority	Nathan Lewis retains definition, canon, and ONESTAMP authority.
Research status	DRAFT — UNDER REVIEW — NOT CANON — ONESTAMP PENDING
Source rule	Verbatim source material is preserved separately from interpretation, proposed definitions, and external research.
Safety rule	The project studies failure through bounded models and controlled simulations; it does not provide instructions to compromise real systems or any claimed base reality.
Review rule	Conflicts, dissent, uncertainty, and source limitations remain visible until resolved by the Architect.

How the topic arose

The topic arose while the Architect was reviewing the Master Change Log and Master Development Log and recognized that many named concepts had only definitions, skeletal outlines, or incomplete supporting detail. Conditional Failure at the Root Level (CFTRL) was selected for deliberate expansion because it combines systems design, nested dependencies, failure propagation, maintenance, simulation, epistemology, and learning from endings. The immediate need is not to ratify a cosmological claim. It is to build documentation and a research program capable of distinguishing testable engineering questions from simulation-testable propositions and presently unresolved ontological questions.

Verbatim origin prompt

*Can you save this thought for me?
I need you to gather research and analysis for a concept.
Conditional failure at the root level.
The study of root level structures, how they arise, how they fail, how they are maintained. Please include the original documents for this content that were generated.
How do you know the difference between a root level structure and a structure that is not? How can a system distinguish between base reality and something that is not? How can a system determine when a root level failure has occurred or when base reality has been compromised.
How do we study this? What about this concept works, what doesn't? What questions do we need to ask?*

*How can we study what lies beyond these structures? Can we model whether structures in reality follow a similar design? Or diverges in ways we haven't thought of? Is this feasible using simulations or another type of modeling? How? Can we design a laboratory environment to do this? How? Can you build a reliable map of reality? What basis of comparison do you have when you are operating on incomplete information? Use failure as an example to learn from.
What I do not assume in the design of this:
That I have the complete architecture and map of all simulations, of reality, of existence as we know it.
That we share a unified definition of what reality is.
That existence can be fully mapped, represented, comprehended in full, or that this is the aim of this laboratory.
That I aim to cause a root level compromise of simulations, of base reality, or any other structure.
That this is a how to guide to do any of those things.
It is intended to help in the design of simulation and reality level structure lifecycle design, maintenance, evolution, and improvement.
Within this design frame It does not assume that we have complete knowledge or understanding of what that is.
How This topic arose from me thinking about conditional failure at the root level The master change log and master development log. I realized that there are a lot of concepts within that framework that have a definition or a name and a skeleton attached to them. They are a rough draft, they are an incomplete outline, there are a lot of additional details and supporting information that need to be added. The goal here is to design documentation to help support, design, and develop this concept. What I will include are the responses of all of the other AI collaboration team members, where they have generated documentation, I will include the documentation, and then I will include this complete response as well.*

Purpose and goal

Purpose. Establish a traceable, evidence-supported outline for designing, developing, testing, governing, and documenting CFTRL without erasing the original source voice or presenting unapproved interpretation as canon.

Goal. Move CFTRL from a strong conceptual skeleton to a reviewable research framework with explicit definitions, system boundaries, failure classes, evidence standards, laboratory methods, metrics, safety controls, team roles, and approval gates. The target is a framework that can support engineered systems and narrative architecture while remaining honest about what cannot be inferred about ultimate base reality.

Program destination

The intended destination is a versioned CFTRL framework that can be used to design and review nested systems, diagnose inherited versus local failure, test resilience before deployment, preserve evidence when foundational services fail, and learn from terminal scenarios. It must remain useful even if no claim about ultimate base reality can be settled.

The program will not treat research volume as progress. Advancement requires controlled terms, traceable requirements, executable models, repeatable experiments, validated measures, documented dissent, resolved safety concerns, and explicit Architect approval. NASA verification and validation practice supports separating conformance to specification from fitness for intended use [R10]; NIST similarly treats resilience as a life-cycle engineering objective [R3].

Design principles

- Boundary before root: every claim identifies the system of interest, external context, function, and analysis horizon [R18].
- Function before label: foundational status is demonstrated through dependency, authority, persistence, and substitution evidence.
- Evidence before attribution: anomaly, model mismatch, shared upstream failure, root failure, and ontological claim are separate conclusions.
- Independent observation: critical evidence and recovery paths should not share every failure domain with the system they assess.
- Resilience by design: systems should anticipate, withstand, recover, and adapt rather than rely on perfect prevention [R3].
- Controlled failure learning: experiments use hypotheses, baselines, limited blast radius, stop conditions, and documented remediation [R13].
- Plural causation: fault trees and causal-factor analysis preserve interacting conditions rather than force one villain component [R11][R20].
- Canon discipline: AI synthesis, external research, proposals, test results, and approved canon remain visibly distinct.

Integrated workstreams

Workstream	Primary output	Lead function	Dependencies
WS1 — Source and provenance	Immutable archive, checksums, source hierarchy, prompt record, conflict log.	Documentation/provenance lead.	Architect confirms authoritative inputs.
WS2 — Ontology and glossary	Controlled definitions for root, conditional, failure, compromise, maintenance, base reality, witness, and invariant.	Concept/formalization lead.	WS1.
WS3 — Systems architecture	Boundary template, dependency/authority/observability model, reference architecture.	Systems engineering lead.	WS2.
WS4 — Failure and evidence	Failure taxonomy, evidence ladder, diagnosis protocol, fault trees.	Reliability/safety lead.	WS2–WS3.
WS5 — Epistemology	Testability boundaries, alternative-model discipline, base-reality claims policy.	Epistemology/methods lead.	WS2 and external research.
WS6 — Simulation laboratory	MVP harness, ground truth, agents, fault injector, witness, experiment registry.	Simulation/software lead.	WS3–WS4.
WS7 — Validation	Benchmarks, metrics, replication, sensitivity, negative results.	Independent verification lead.	WS6.
WS8 — Department of Endings	Charter, runbook, reporting, design-diff workflow, oversight.	Governance/operations lead.	WS4–WS7.
WS9 — Narrative integration	Use cases and scenes that obey approved system rules.	Narrative architecture lead.	Approved outputs from WS2–WS4.
WS10 — Canon and release	Review packet, dissent register, decision log, ONESTAMP candidate.	Architect and records custodian.	All prior workstreams.

Phased roadmap

Phase	Objective	Required outputs	Exit gate
0 — Source freeze	Prevent loss, drift, and contamination.	Archive; hashes; inventory; verbatim prompt; source-status labels.	Architect confirms corpus completeness.
1 — Definition baseline	Make the concept reviewable.	Purpose; scope; non-goals; glossary; system-boundary template; claims ledger.	No unresolved term blocks model construction.
2 — Formal framework	Translate language into a model.	Rootness vector; layer graph; authority/observability edges; invariants; failure-state model.	Peer review shows definitions are internally consistent.
3 — Experiment design	Specify how claims can fail.	Hypotheses; falsifiers; experiment matrix; metrics; safety limits; data schema.	Independent reviewer can reproduce planned trials from protocol.
4 — MVP laboratory	Demonstrate the central phenomenon.	Root Identification Blind Test; baseline; local/root/telemetry faults; ground-truth scorer.	Repeatable misdiagnosis and correct-diagnosis results under controlled seeds.
5 — Expansion and stress	Test scale, mixed modes, and architectures.	Quorum roots; Byzantine behavior; partitions; objective drift; redundancy; scale sweeps.	Sensitivity analysis identifies where conclusions hold and fail.
6 — External validation	Compare with known disciplines and cases.	Virtualization, SRE, infrastructure, safety, and organizational case mappings.	Claims survive cross-domain review without overgeneralization.
7 — Operationalization	Create the learning institution.	Department of Endings charter; intake; exercise; report; remediation verification.	At least one finding becomes a verified design change.
8 — Canon review	Prepare approved release.	Integrated dossier; dissent register; unresolved questions; release notes.	Architect ONESTAMP or returns changes.

Suggested sequencing

The schedule below is a planning baseline, not a commitment. Work can overlap only where traceability is maintained. The minimum path to an informative result is approximately 12–18 focused weeks; a validated multidisciplinary framework will take longer and should not be rushed.

Window	Priority	Concrete outcome
Days 0–30	Source freeze, glossary, boundary template, claims ledger.	A stable definition package and explicit list of unresolved decisions.
Days 31–60	Formal model, taxonomy, evidence ladder, experiment protocol.	A simulation-ready specification with acceptance criteria.
Days 61–90	MVP Root Identification Blind Test and baseline trials.	First reproducible evidence about local-versus-root classification.
Months 4–6	Mixed-mode failures, witness designs, redundancy variants, validation.	Characterized limits and initial design guidance.
Months 7–12	Department of Endings pilot, external analogues, independent replication.	Operational process and candidate framework release.

Decision gates

Gate	Decision owner	Questions	Evidence required
G0 Corpus accepted	Architect	Are all originals present and correctly labeled?	Inventory, hashes, provenance notes.
G1 Definition accepted	Architect with team review	Are terms clear enough to model without hidden assumptions?	Glossary, examples, counterexamples, dissent.
G2 Model accepted	Systems and methods reviewers	Does the model represent dependency, authority, observability, and uncertainty?	Formal schema and worked cases.
G3 Experiment authorized	Safety and technical reviewers	Are hypotheses falsifiable and interventions bounded?	Protocol, stop conditions, risk review.
G4 MVP validated	Independent verification	Can results be reproduced and are metrics calibrated?	Seeds, code, logs, result package.
G5 Operational pilot approved	Architect/governance	Can the Department of Endings act without becoming an unchecked root?	Charter, oversight, escalation, audit plan.
G6 Canon candidate	Architect	What is approved, rejected, deferred, or still unknown?	Integrated release, dissent register, source traceability.

Team operating model

Human authority

The Architect retains authority over concept intent, organizational meaning, canon, and ONESTAMP. Research contributors may propose definitions, tests, objections, and revisions; they may not silently convert inference into authorial intent. Safety and verification reviewers must be able to block an experiment or claim from advancing without blocking preservation of the idea itself.

Functional roles

Role	Accountability	Required work product
Architect / project owner	Intent, scope, priority, final decisions, canon.	Decision record and ONESTAMP action.
Program integrator	Coordinates dependencies, schedule, gates, and unresolved decisions.	Integrated roadmap and status report.
Research lead	Finds and evaluates external evidence; separates primary from secondary sources.	Evidence matrix and literature synthesis.
Systems architect	Defines boundaries, functions, interfaces, dependency and authority graphs.	Reference architecture and model schema.
Simulation lead	Implements deterministic, reproducible laboratory infrastructure.	Code, configuration, seeds, runbook.
Reliability/safety lead	Defines hazards, fault trees, containment, stop conditions, and recovery tests.	Safety case and failure taxonomy.
Epistemology/methods lead	Guards inference, falsifiability, alternative models, and uncertainty.	Claims ledger and testability review.
Independent verifier	Reproduces experiments and challenges measures.	Verification report and discrepancy log.
Records/provenance custodian	Preserves originals, versions, hashes, changes, and approval status.	Source archive and release manifest.
Red team / Department of Endings	Designs adverse scenarios and tests assumptions.	Experiment proposals and lessons-to-design diffs.
Narrative architect	Applies approved rules to scenes and worldbuilding without rewriting technical doctrine.	Use cases and narrative conformance review.

Provisional AI collaboration map

The existing corpus includes contributions from ChatGPT, Claude, Gemini, Copilot, Perplexity, and Grok. The mapping below is a coordination proposal based on the form of their existing contributions, not a permanent assignment or evaluation of capability. Roles should be rotated or duplicated to reduce single-model dependence.

Collaborator	Provisional contribution lane	Required constraint
ChatGPT	Framework synthesis, formal definitions, experiment questions.	Trace every proposed definition to source or label it as new.
Claude	Long-form critique, safety/ethics review, contradiction finding.	Separate editorial confidence from verified evidence.
Gemini	Cross-domain scenarios and alternative structures.	Exclude unrelated personal-log content and unsupported cosmological certainty.
Copilot	Implementation planning, code review, laboratory automation.	No production targeting; reproducibility and tests required.
Perplexity	External research, source verification, provenance, integrated reporting.	Primary sources first; no fabricated citations or canon claims.
Grok	Adversarial synthesis, stress cases, unresolved tensions.	Verify remembered bibliography before public citation.
Model council	Compare answers, preserve dissent, identify convergence and assumption differences.	Do not use majority vote as truth or canon.

Perplexity AI support plan

Perplexity AI will support the team as the evidence-and-integration layer rather than as the authority that defines canon. The service plan is designed to help every collaborator work from the same source record and to prevent research claims, generated interpretation, and Architect decisions from becoming mixed.

Maintain a source register that links each requirement, definition, and open question to the exact originating prompt or document.

Research adjacent disciplines using primary and authoritative sources, then state what each source supports and what it does not support.

Produce crosswalks that compare AI-team proposals, identify agreement, isolate contradictions, and preserve minority objections.

Convert approved concepts into testable requirements, experiment hypotheses, measures, and review checklists without silently changing intent.

Create periodic project assessments showing completed evidence, unresolved decisions, risks, and the next gate.

Validate citations, dates, quotations, and external claims before release; quarantine unverifiable remembered references.

Assist the simulation team with experiment matrices, data schemas, reproducibility manifests, and result interpretation.

Assist the Department of Endings with pre-mortem research, real-world analogues, and lessons-to-design traceability.

Prepare DOCX/PDF review packets and preserve separate verbatim, research, proposed, and approved layers.

Never apply ONESTAMP, assert authorial approval, or elevate a draft to canon; only the Architect can do so.

Collaboration protocol

Rule	Implementation
One question per task packet	Break compound prompts into definition, evidence, design, critique, and decision requests.
Source-first context	Attach the verbatim prompt and relevant originals before asking for synthesis.
Claim labels	Mark content as SOURCE, EXTERNAL EVIDENCE, ANALYSIS, PROPOSAL, TEST RESULT, DECISION, or CANON.
Dissent preservation	Record unresolved disagreement and strongest counterargument; do not smooth it away.
No silent merges	Keep collaborator outputs intact and create a separate synthesis document.
Traceability	Every requirement and approved term links backward to a source and forward to a test or decision.
Reproducibility	Every simulation result includes code version, configuration, seed, environment, and raw output hash.
Review independence	The model that generates a claim should not be the only model reviewing it.
Change control	Substantive revisions receive version, rationale, impact assessment, reviewer, and decision status.
Release discipline	Only approved packages use canonical identifiers; drafts retain explicit pending status.

Documentation architecture

Document	Purpose	Status owner
Source Archive	Untouched prompts, original AI outputs, original generated files, checksums.	Records custodian; immutable.
Requirements and Intent Record	What the Architect asked, did not assume, and intends to achieve.	Architect.
Research Dossier	External evidence, synthesis, limitations, competing views.	Research lead; always distinguish evidence from proposal.
Framework Specification	Controlled glossary, boundary model, taxonomy, evidence protocol.	Systems/methods leads; candidate for approval.
Laboratory Specification	Architecture, experiment registry, data schema, metrics, safety controls.	Simulation and safety leads.
Experiment Reports	Pre-registration, execution record, results, anomalies, interpretation.	Experiment owner plus independent verifier.
Department of Endings Charter	Mandate, limits, workflows, decision rights, audit and escalation.	Architect/governance.
Decision and Dissent Log	Accepted, rejected, deferred, unresolved, and reasons.	Program integrator.
Canonical Release Package	Approved text, release notes, source traceability, ONESTAMP record.	Architect and records custodian.

Requirements traceability

Each requirement should have: unique ID, source citation, rationale, owner, verification method, validation context, status, and change history. NASA's verification/validation distinction provides a useful structure [R10]. A definition can be verified for consistency with the specification; the framework must still be validated by showing that it helps users diagnose, design, or learn under representative conditions.

Requirement ID	Requirement	Verification	Validation
CFTRL-R-001	Declare system and analysis boundaries before assigning root status.	Document inspection.	Independent reviewers reach consistent boundary interpretation.
CFTRL-R-002	Separate operational-root claims from ontological-base claims.	Claims-ledger audit.	No experiment report converts E0–E4 evidence into an E5 claim.
CFTRL-R-003	Preserve at least one evidence channel outside the primary mutable trust chain.	Architecture inspection and fault injection.	Decisive evidence survives selected root/evidence failures.
CFTRL-R-004	Report confidence and viable alternatives with every diagnosis.	Schema validation.	Confidence is empirically calibrated across trials.
CFTRL-R-005	Use bounded experiments with stop conditions.	Safety-plan review.	Exercises complete without escaping declared scope.
CFTRL-R-006	Convert findings into tracked design changes and rerun tests.	Traceability audit.	Verified recurrence reduction or improved containment.

Department of Endings

Proposed charter

Mission. Ensure that credible failure and ending conditions are examined before they become unplanned reality and that observed failures produce verified design learning. This mission aligns with safety-management cycles that collect data, identify risk, implement controls, and evaluate effectiveness [R17], but it must be adapted to OAL's narrative and simulation context.

Charter element	Proposed rule
Mandate	Design and run bounded failure studies; preserve outcomes; issue design findings; verify corrective action.
Non-mandate	No authority to create canon, cause uncontrolled real-world failure, or declare metaphysical truth.
Intake	Accept architecture reviews, incident records, unresolved assumptions, and Architect-directed endgames.
Products	Pre-mortem, experiment plan, run record, failure map, misleading-signals list, lesson, design diff, verification result.
Escalation	Immediate stop and human review for safety breach, uncontrolled propagation, data integrity loss, or dual-use concern.
Oversight	Architect authority plus independent safety and provenance review.
Success measure	Not number of disasters imagined; percentage of significant findings converted into verified design improvement.
Anti-capture	Rotate reviewers, preserve dissent, audit false alarms, and publish null results.

Department workflow

Receive a system boundary, purpose, requirements, dependency map, and unresolved assumptions.

Identify top events and causal combinations using fault-tree and event-chain methods [R11][R20].

Prioritize scenarios by consequence, plausibility, uncertainty, learning value, and reversibility.

Pre-register hypothesis, baseline, intervention, measures, stop conditions, and expected discriminating evidence.

Run synthetic or staged experiments with independent monitoring and evidence preservation.

Classify outcomes: confirmed, disconfirmed, inconclusive, invalid test, or new hazard discovered.

Issue a design diff with owner, due date, acceptance criterion, and required retest.

Verify the corrective action, record residual risk, and update the framework only through change control.

Laboratory implementation plan

Build increment	Scope	Acceptance criterion
MVP-0	Deterministic stack, no faults, complete ground-truth log.	Repeated seed produces identical output.
MVP-1	Local and parent resource faults.	Agents can report class, layer, confidence, and evidence.
MVP-2	False telemetry and independent witness.	System demonstrates both successful and failed attestation cases.
MVP-3	Partitions and delayed messages.	Reports distinguish crash, slowness, and uncertainty under stated assumptions.
MVP-4	Compromised objective and Byzantine reports.	Change detection and equivocation evidence are scored.
MVP-5	Redundancy architectures and scale sweep.	Sensitivity and common-mode failure results are reproducible.
MVP-6	External benchmark cases.	Framework explains known engineering failures without claiming more than evidence supports.

Risk register

Risk	Consequence	Control	Owner
Ontological overclaim	Speculation presented as scientific finding.	Evidence ladder; mandatory testability review.	Methods lead.
Source drift	AI synthesis replaces Architect wording.	Verbatim archive; claim labels; traceability.	Records custodian.
Citation fabrication	Public credibility loss.	Primary-source verification and URL check.	Research lead.
Model confirmation bias	Lab proves only assumptions built into it.	Blind trials, alternative models, adversarial review, null publication.	Independent verifier.
Common-mode instrumentation	Root corrupts detector and record.	Independent witness and failure-domain audit.	Systems architect.
Unsafe experimentation	Uncontrolled or dual-use impact.	Synthetic-first, scope controls, stop conditions, review.	Safety lead.
Department capture	Endings team becomes authority theater or bottleneck.	Limited charter, transparent metrics, rotating review.	Governance lead.
Complexity inflation	Framework becomes too elaborate to use.	Minimum viable schema; user validation; retire unused elements.	Program integrator.
AI consensus illusion	Agreement mistaken for truth.	Preserve dissent and evidence quality; no majority canon.	Architect.
Narrative/technical leakage	Scene choices silently redefine system rules.	Conformance review and separate documents.	Narrative architect.

Near-term task board

Priority	Task	Output	Definition of done
P0	Approve source inventory and contamination note.	Frozen corpus manifest.	All originals hashed; unrelated content flagged.
P0	Decide canonical meaning of “root” for first model.	Decision record.	Selected operational root type and excluded meanings.
P0	Approve non-goals and safety boundary.	Scope statement.	No real-world compromise instructions; no ontological proof claim.
P1	Review glossary and rootness vector.	Framework v0.2.	Examples and counterexamples accepted.
P1	Select first four failure classes.	Experiment backlog.	Each has falsifier, metric, and stop condition.
P1	Define legitimate-state invariants.	Invariant catalog.	Machine-readable and human-reviewed.
P2	Implement MVP-0 and MVP-1.	Runnable repository.	Deterministic runs and baseline report.
P2	Draft Department charter.	Charter v0.1.	Authority, limits, products, oversight defined.
P2	Create review cadence.	Calendar and meeting template.	Decision owner and evidence packet specified for each gate.

Success criteria

- The team can distinguish source text, research evidence, proposed framework language, experiment result, Architect decision, and canon at a glance.
- Two independent reviewers can apply the root classification method to the same bounded system and explain disagreements.
- The laboratory reproduces local-fault, inherited-fault, telemetry-fault, and no-fault trials and accurately records ground truth.
- Diagnostic agents improve calibration and reduce harmful local repairs when cross-layer evidence is available.
- At least one proposed “root signature” is disconfirmed or narrowed, demonstrating that the process can correct itself.
- Department of Endings findings produce traceable design changes followed by verification, not only reports.
- Ontological uncertainty remains explicit; no operational experiment is represented as proof of ultimate base reality.
- A complete candidate release can be reviewed by the Architect with unresolved decisions and dissent visible.

Conclusion

The team reaches the target by progressing through evidence gates, not by merging more prose. The first program milestone is a controlled definition package; the first technical milestone is a blind experiment that can distinguish local and inherited failure under known ground truth; the first institutional milestone is a Department of Endings finding that becomes a verified design improvement. Perplexity’s role is to keep evidence, provenance, cross-team synthesis, and documentation coherent while leaving intent and canon authority with the Architect.

Research evidence register

The following sources support the research layer. They are external evidence, not OAL canon. Inline markers [R#] identify the source supporting a claim. Source descriptions are intentionally limited to what the cited material establishes.

ID	Source	Relevance	URL
R1	NIST CSRC, "Trusted Computing Base (TCB)"	Defines the TCB as the totality of hardware, firmware, and software protection mechanisms responsible for enforcing a security policy.	https://csrc.nist.gov/glossary/term/trusted_computing_base
R2	NIST, Hardware-Enabled Security / Platform Integrity	Describes cryptographic measurement, chains of trust, roots of trust for update, detection, and recovery, and remote attestation.	https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=934427
R3	NIST SP 800-160 Vol. 2 Rev. 1, Developing Cyber-Resilient Systems	Frames resilience as anticipating, withstanding, recovering from, and adapting to adverse conditions, stresses, attacks, and compromises across the system life cycle.	https://csrc.nist.gov/pubs/sp/800/160/v2/r1/final
R4	Google SRE, "Addressing Cascading Failures"	Shows how overload, feedback, and secondary symptoms can spread failures and make apparent causes misleading.	https://sre.google/sre-book/addressing-cascading-failures/
R5	Buldyrev et al., Nature 464 (2010), "Catastrophic cascade of failures in interdependent networks"	Demonstrates that dependency coupling can propagate small failures recursively across networks and produce fragmentation.	https://www.nature.com/articles/nature08932
R6	E. W. Dijkstra, "Self-stabilizing Systems in Spite of Distributed Control" (1974)	Defines self-stabilization as eventual return to a legitimate state regardless of initial state, under the stated model.	https://courses.csail.mit.edu/6.852/05/papers/p643-Dijkstra.pdf
R7	Lamport, Shostak, and Pease, "The Byzantine Generals Problem" (1982)	Formalizes agreement when components may send conflicting information and establishes fault-tolerance limits under specified communication assumptions.	https://dl.acm.org/doi/pdf/10.1145/357172.357176
R8	Fischer, Lynch, and Paterson, "Impossibility of Distributed Consensus with One Faulty Process" (1985)	Shows that deterministic consensus cannot be guaranteed in a fully asynchronous system even with one crash fault under the paper's assumptions.	https://www.cs.princeton.edu/courses/archive/fall07/cos518/papers/flp.pdf
R9	Stanford Encyclopedia of Philosophy, "Causal Models"	Explains that causal models support predictions about interventions and counterfactuals, not only observed correlations.	https://plato.stanford.edu/entries/causal-models/
R10	NASA Systems Engineering Handbook / Expanded Guidance	Separates verification ("built right") from validation ("right product") and requires planning, execution, analysis, reporting, and work-product capture.	https://www.nasa.gov/reference/systems-engineering-handbook/
R11	NASA Software Engineering Handbook, "Software Fault Tree Analysis"	Describes deductive analysis from an undesired top event to credible combinations of contributing faults or system states.	https://swehb.nasa.gov/spaces/SWEHBVC/pages/72025030/8.07+-+Software+Fault+Tree+Analysis
R12	CISA, Shields Ready	Recommends identifying critical assets, mapping dependencies, assessing risks, exercising plans, and adapting from lessons learned.	https://www.cisa.gov/shields-ready
R13	AWS Prescriptive Guidance, Chaos Engineering	Distinguishes deterministic resilience testing from hypothesis-based chaos experiments; emphasizes baselines, stop conditions, limited blast radius, monitoring, and learning.	https://docs.aws.amazon.com/prescriptive-guidance/latest/chaos-engineering-on-aws/overview.html
R14	Nick Bostrom, "Are We Living in a Computer Simulation?" (2003)	Presents a probabilistic trilemma; it does not itself provide a direct internal test for identifying base reality.	https://academic.oup.com/pq/article-abstract/53/21/1/243/1610975
R15	David Chalmers, discussion of perfect and imperfect simulations (2022)	Argues that an imperfect simulation might provide detectable evidence, while a perfect simulation may be observationally indistinguishable from non-simulated reality.	https://www.carnegiecouncil.org/media/series/aiei/how-real-is-virtual-reality-david-chalmers
R16	Vazza, "Astrophysical constraints on the simulation hypothesis" (Frontiers in Physics, 2025)	Evaluates physical resource constraints for particular simulation scenarios and emphasizes that claims outside shared physical assumptions remain difficult or impossible to falsify.	https://www.frontiersin.org/journals/physics/articles/10.3389/fphy.2025.1561873/full
R17	NTSB, Safety Management Systems	Treats safety as an organization-wide cycle of policy, risk management, assurance, promotion, data collection, mitigation, and evaluation.	https://www.nts.gov/Advocacy/mwl/Pages/mwl-3.aspx
R18	INCOSE definition and systems-boundary discussion (IET summary)	Defines systems engineering as an integrative life-cycle approach and distinguishes the system boundary from its broader context.	https://www.theiet.org/impact-society/policy-and-public-affairs/digital-futures-policy/reports-and-papers/quantum-technologies-a-systems-approach

ID	Source	Relevance	URL
R19	NIST SP 800-207, Zero Trust Architecture	Rejects implicit trust based only on location or ownership and requires explicit authentication and authorization for access.	https://www.nist.gov/publications/zero-trust-architecture
R20	NASA Spaceflight Mishap Investigation Handbook	Uses timelines, fault trees, event-and-causal-factor trees, and recommendations to identify chains of events and conditions rather than stopping at a proximate cause.	https://www.nasa.gov/wp-content/uploads/2023/03/spaceflight-mishap-handbook-tagged.pdf